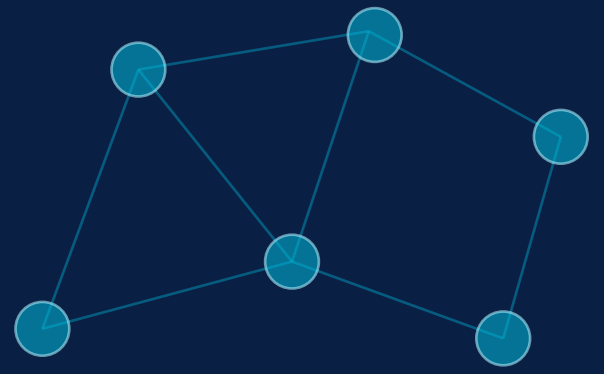
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

CA3 PRESENTATION

DESIGN AND ANALYSIS OF ALGORITHMS

TOPIC: MERGE SORT



Presented By	Nehal Buxy
Semester	III
Section	D
PRN	25070521185
Guided By	Dr. Smita Singh Ma’am

Introduction, Divide and Conquer, and Algorithm

Merge Sort is a **comparison-based** sorting algorithm: it orders elements only by comparing pairs of keys. It follows the Divide and Conquer paradigm.

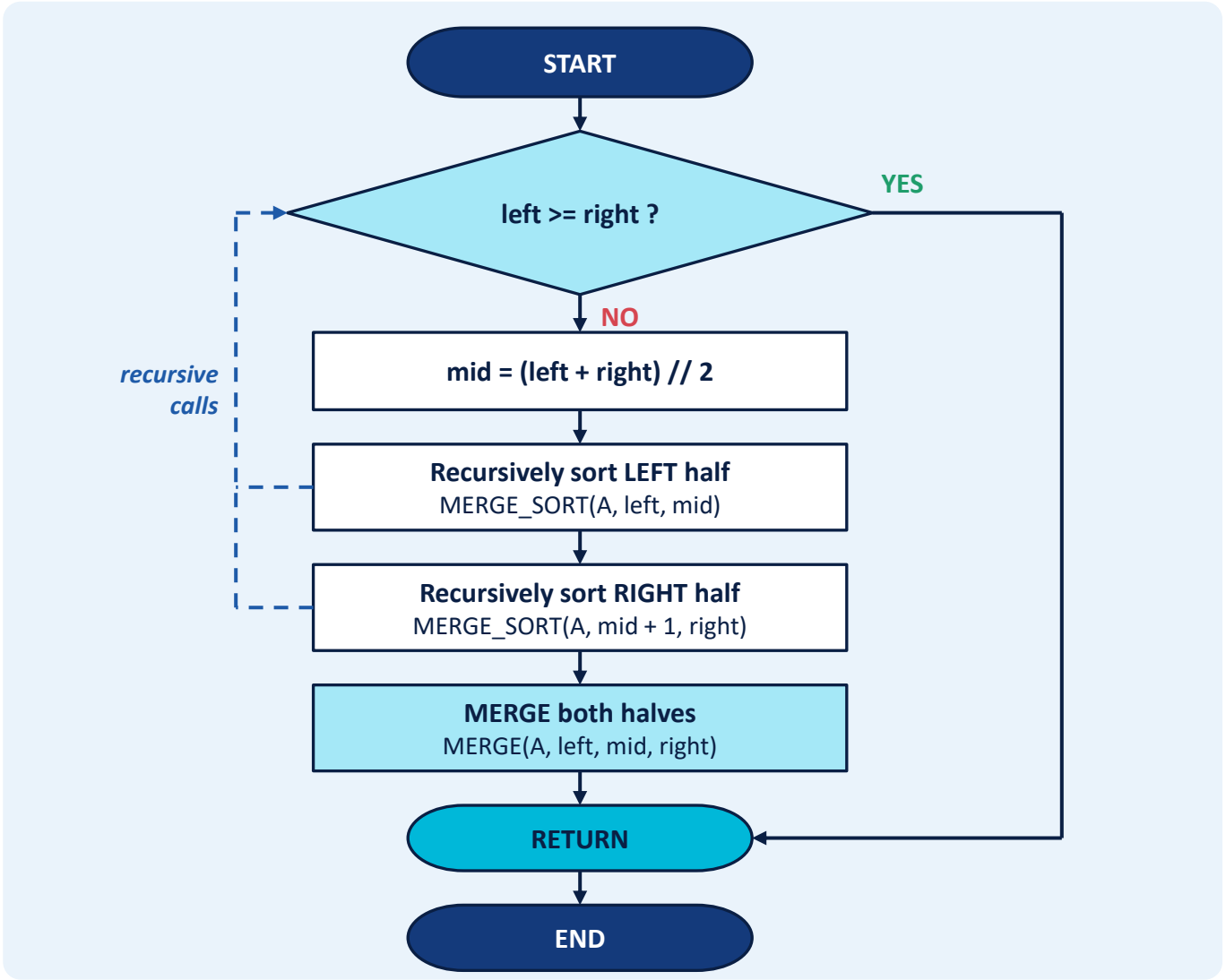
1 DIVIDE
Split the array into smaller subarrays.

2 CONQUER
Recursively sort each subarray.

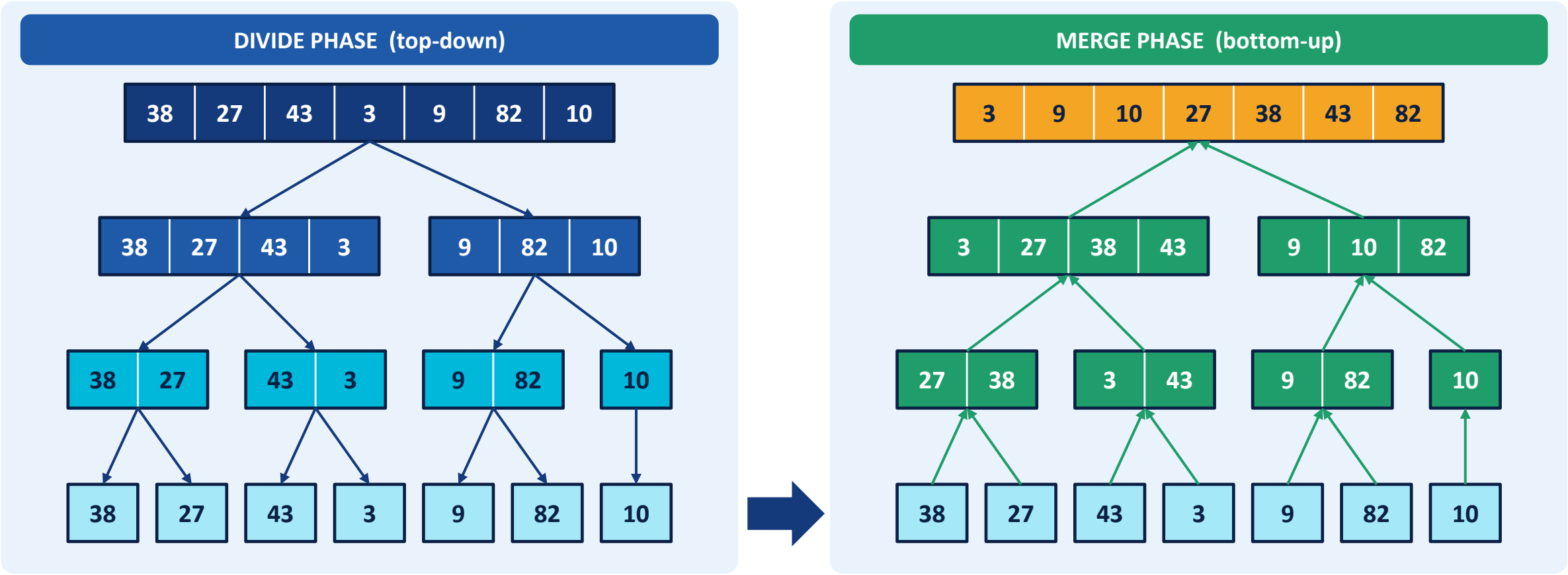
3 COMBINE
Merge sorted subarrays into one sorted array.

Base case: an array with zero or one element is already sorted.

```
MERGE_SORT(A, left, right)
1. If left >= right, return.
2. mid = (left + right) // 2
3. MERGE_SORT(A, left, mid)
4. MERGE_SORT(A, mid + 1, right)
5. MERGE(A, left, mid, right)
```



Detailed Working with Example



FINAL SORTED ARRAY 3 9 10 27 38 43 82

Merge rule: compare the front elements of both sorted halves and copy the smaller first; when one half is empty, copy the rest.

Complexity, Advantages, Disadvantages and Applications

Case	Time	Space (auxiliary)
Best Case	O(n log n)	O(n)
Average Case	O(n log n)	O(n)
Worst Case	O(n log n)	O(n)

Space: O(n) auxiliary for a standard array-based implementation.

WHY O(n log n)?

- The array is divided into approximately $\log_2(n)$ levels.
- Each level requires O(n) total merging work.
- Therefore, the total time complexity is O(n log n).

$$\log_2 n \text{ levels} \times O(n) \text{ per level} = O(n \log n)$$

+ ADVANTAGES

- Predictable O(n log n) running time.
- Stable sorting when equal elements are merged appropriately.
- Suitable for linked lists and external sorting of large datasets.
- Effective for sorting large datasets.

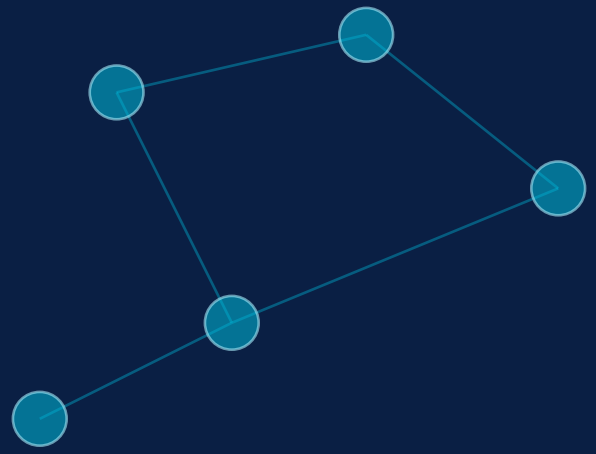
– DISADVANTAGES

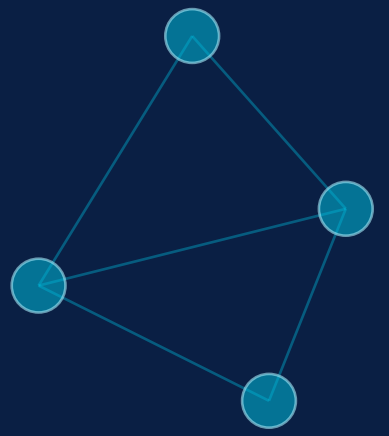
- Standard array-based implementation requires O(n) auxiliary memory.
- Recursive calls introduce overhead.
- Often less memory-efficient than in-place algorithms such as Quick Sort for arrays.

APPLICATIONS

- **External sorting of large files**
- **Sorting linked lists**
- **Stable sorting of records**
- **Parallel sorting implementations**

Generated using Claude





THANK YOU

CA3 – Design and Analysis of Algorithms

Presented by Nehal Buxy | Semester III | Section D